

# OpenMower-TAF-ROS

## Änderungsdokumentation - Checkpoint nach Planerstellung

|                               |                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------|
| <b>Projektname</b>            | OpenMower-TAF-ROS                                                               |
| <b>Dokumentenart</b>          | Änderungsdokumentation / Codepaket-Dokumentation                                |
| <b>Thema</b>                  | checkpoint.bag wird zusätzlich nach erfolgreicher Mähplanerstellung geschrieben |
| <b>Erstellungsdatum</b>       | 2026-07-03                                                                      |
| <b>Änderungsdatum</b>         | 2026-07-03                                                                      |
| <b>Version</b>                | v0.1                                                                            |
| <b>Status</b>                 | Entwurf                                                                         |
| <b>Autor / verantwortlich</b> | OpenAI ChatGPT, fachliche Vorgabe durch Projektanwender                         |
| <b>Ablageort</b>              | OpenMower-TAF-ROS/_Dokumentation und bereitgestellte Download-Artefakte         |
| <b>Referenzen</b>             | OpenMower-TAF-ROS.zip;<br>Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf     |

### 1. Ziel der Änderung

Ziel ist, die Wiederaufnahmeinformationen früher und robuster zu persistieren. Bisher wurde checkpoint.bag erst bei reset(), bei skip\_path oder während des aktiven Mähens periodisch geschrieben. Nach dieser Änderung wird checkpoint.bag zusätzlich direkt nach erfolgreicher Erstellung und Initialisierung eines neuen Mähplans für eine Fläche geschrieben.

Damit enthält checkpoint.bag bereits die aktive area\_id, die Plan-Metadaten und den initialisierten aktuellen Pfad, bevor der Roboter den ersten periodischen 30-Sekunden-Checkpoint während der Pfadausführung erreicht.

### 2. Fachliches Verhalten vorher / nachher

| Aspekt                           | Vorher                                                                                           | Nachher                                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Planerstellung</b>            | currentAreald und checkpointAreald wurden im RAM gesetzt.                                        | currentAreald und checkpointAreald werden weiterhin gesetzt.                                                           |
| <b>checkpoint.bag</b>            | Wurde nicht direkt nach erfolgreicher Planerstellung geschrieben.                                | Wird zusätzlich direkt nach erfolgreicher Planerstellung und Pfadinitialisierung geschrieben.                          |
| <b>Bestehende Speicherpunkte</b> | reset(), skip_path und periodischer Checkpoint während des Mähens.                               | Unverändert: alle bisherigen Speicherpunkte bleiben zusätzlich bestehen.                                               |
| <b>MQTT / Robot State</b>        | checkpoint_area_id konnte nach Neustart vom letzten gespeicherten checkpoint.bag-Stand abhängen. | checkpoint_area_id kann bereits nach Planerstellung aus dem neu geschriebenen checkpoint.bag wiederhergestellt werden. |

### 3. Geänderte Datei

|                         |                                                                                                               |
|-------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Datei</b>            | src/mower_logic/src/mower_logic/behaviors/MowingBehavior.cpp                                                  |
| <b>Funktion</b>         | bool MowingBehavior::create_mowing_plan(int area_index)                                                       |
| <b>Art der Änderung</b> | Zusätzlicher checkpoint()-Aufruf nach start_current_mowing_plan_path() und vor publish_mowing_progress(true). |

## 4. Technische Umsetzung

Der neue checkpoint()-Aufruf liegt bewusst erst am Ende der erfolgreichen Planerstellung. Zu diesem Zeitpunkt wurde der Mähplan erzeugt, optional optimiert, gespeichert, der Digest geprüft beziehungsweise aktualisiert und der aktuelle Pfad über start\_current\_mowing\_plan\_path() initialisiert.

Eingefügter Codeausschnitt:

```
start_current_mowing_plan_path();

// Persist the freshly created and selected mowing plan immediately.
// This makes checkpoint.bag contain the active area_id and plan metadata before
// the mower reaches the first periodic checkpoint during path execution.
checkpoint();
ROS_INFO_STREAM("MowingBehavior: Saved checkpoint after successful mowing plan creation for
area_id="
                << currentAreaId << " plan_id=" << currentMowingPlan.plan_id);

publish_mowing_progress(true);
return true;
```

## 5. Warum diese Stelle gewählt wurde

Ein früherer checkpoint()-Aufruf direkt nach currentAreald = selected\_area\_id wäre technisch möglich, aber fachlich weniger sauber. An dieser frühen Stelle ist noch nicht sichergestellt, dass der Plan vollständig erzeugt, gespeichert und als aktueller Pfad initialisiert wurde.

Die gewählte Stelle verhindert, dass checkpoint.bag nur eine Area-ID ohne belastbare Plan-Metadaten enthält. Dadurch passt der Checkpoint besser zum tatsächlich verwendbaren Wiederaufnahmestand.

## 6. Persistierte Inhalte im neuen Zeitpunkt

Beim zusätzlichen Schreibvorgang werden dieselben Felder wie bei den bestehenden checkpoint()-Aufrufen geschrieben, darunter:

- currentMowingAreald: aktive Fläche / area\_id
- currentMowingArea: aktueller Queue-Index
- currentMowingPath und currentMowingPathIndex: initialer oder wiederhergestellter Pfadstand
- currentMowingPlanDigest: Digest des aktuellen Mähplans
- plan\_id und plan\_file: gespeicherte Planidentität und Pfad zur Plan-Datei
- current\_path\_id und current\_pose\_index: initialisierter aktiver Pfad
- mowing\_area\_mode, area\_queue\_digest und current\_area\_queue\_index: Queue- und Modus-Metadaten

## 7. Auswirkungen und Kompatibilität

Die Änderung ist additiv. Die Struktur von CheckPoint.msg, die MQTT-Felder und die bestehenden Speicherpunkte werden nicht geändert. Bestehende Clients müssen nicht angepasst werden.

Nebenwirkung: Nach erfolgreicher Planerstellung erfolgt ein zusätzlicher Schreibzugriff auf checkpoint.bag. Da checkpoint.bag bereits als Snapshot geschrieben wird, ersetzt dieser Schreibvorgang den vorherigen Snapshot durch den aktuelleren Stand.

## 8. Prüfung

| Prüfung               | Ergebnis  | Bemerkung                                                      |
|-----------------------|-----------|----------------------------------------------------------------|
| Statische Codeprüfung | Bestanden | git diff --check ohne Auffälligkeiten für die geänderte Datei. |
| Patchprüfung          | Bestanden | Patch-Datei wurde aus dem tatsächlichen Git-Diff erzeugt.      |
| Paketstruktur         | Bestanden | Root-Ordner OpenMower-TAF-ROS bleibt im ZIP unverändert.       |
| Dokumentation         | Bestanden | PDF und DOCX mit gleichem Stand v0.1 erstellt.                 |

## 9. Manuelle Laufzeitprüfung auf dem Roboter

Empfohlener Test nach Einspielen des Pakets:

```
docker exec -it open_mower_ros bash
source /opt/ros/noetic/setup.bash
# Mahauftrag für eine Fläche starten und im Log prüfen:
# Erwartete Logzeile:
# MowingBehavior: Saved checkpoint after successful mowing plan creation for area_id=...
plan_id=...

# Danach checkpoint.bag prüfen:
rosbag info checkpoint.bag
rosbag echo checkpoint.bag /checkpoint
```

## 10. Änderungshistorie

| Version | Datum      | Bearbeiter     | Status  | Änderung                                                                                                     |
|---------|------------|----------------|---------|--------------------------------------------------------------------------------------------------------------|
| v0.1    | 2026-07-03 | OpenAI ChatGPT | Entwurf | Erste Dokumentation und Codeänderung: checkpoint.bag zusätzlich nach erfolgreicher Planerstellung schreiben. |

## 11. Commit-Nachricht

BL\_Save checkpoint after mowing plan creation

Date: 2026-07-03

Summary:

- Added an immediate checkpoint write after a mowing plan has been successfully created, selected and initialized.
- Ensures checkpoint.bag already contains the active area id and mowing plan metadata before the first periodic path-execution checkpoint.
- Kept the existing periodic, reset and skip-path checkpoint behavior unchanged.

Changed files:

- src/mower\_logic/src/mower\_logic/behaviors/MowingBehavior.cpp

## 12. Patch-Auszug

```
diff --git a/src/mower_logic/src/mower_logic/behaviors/MowingBehavior.cpp
b/src/mower_logic/src/mower_logic/behaviors/MowingBehavior.cpp
index 641b447..6b2531f 100644
--- a/src/mower_logic/src/mower_logic/behaviors/MowingBehavior.cpp
+++ b/src/mower_logic/src/mower_logic/behaviors/MowingBehavior.cpp
@@ -1058,6 +1058,13 @@ bool MowingBehavior::create_mowing_plan(int area_index) {
```

Stand: 2026-07-03 - Status: Entwurf - Ablage: \_Dokumentation

```
}
start_current_mowing_plan_path();

+ // Persist the freshly created and selected mowing plan immediately.
+ // This makes checkpoint.bag contain the active area_id and plan metadata before
+ // the mower reaches the first periodic checkpoint during path execution.
+ checkpoint();
+ ROS_INFO_STREAM("MowingBehavior: Saved checkpoint after successful mowing plan creation for
area_id="
+               << currentAreaId << " plan_id=" << currentMowingPlan.plan_id);
+
+   publish_mowing_progress(true);
+   return true;
}
```